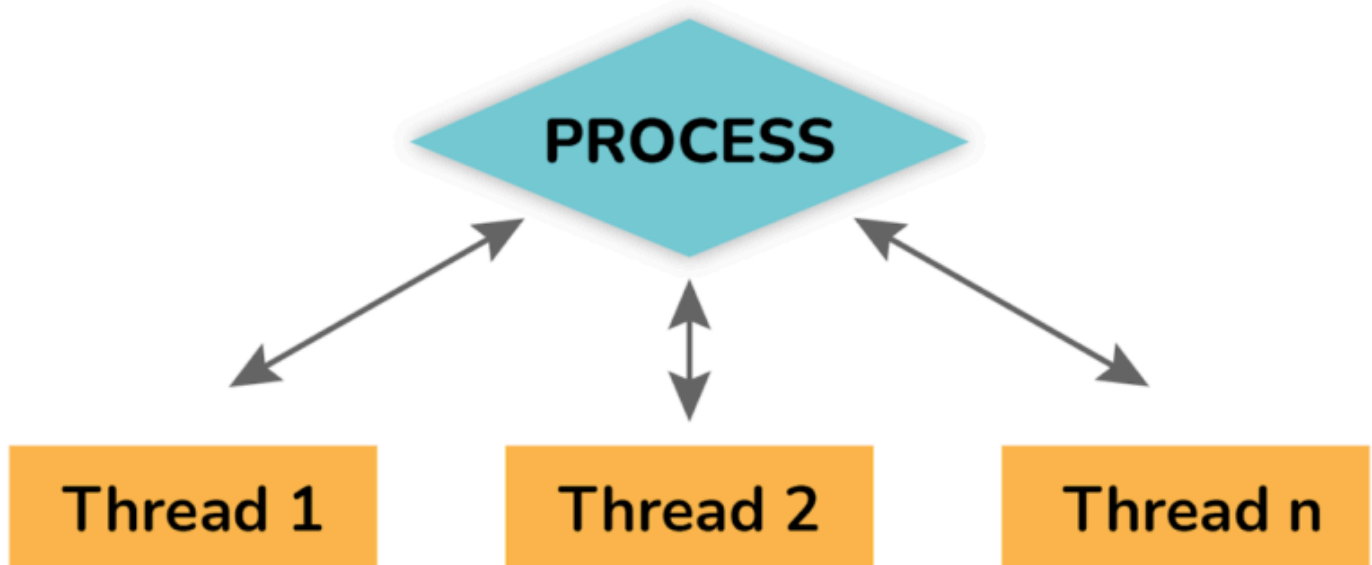


Mutexisation

Paralléliser et multithreader vos processus

Multithreading



ROOTS

Guillaume Bomben

Lyes HAMICI

Helwan-Djibril Mimouni

Oroitz Lago Ramos

Sommaire

I. Introduction

II. Concepts et Documentation

- a. Concepts de base du traitement d'images
- b. Concepts du multi-threading
- c. Bibliothèques utilisées

III. Développement et Implémentation

- a. Implémentation séquentielle
- b. Optimisation avec le multi-threading

IV. Analyse des Performances

- a. Chargement de différentes images
- b. Performances version séquentielle
- c. Performances version multi-thread

V. Résultats et Discussions

- a. Résumé des observations - comparaison

I. INTRODUCTION

Le **traitement d'images** est une discipline clé dans de nombreux domaines tels que la vision par ordinateur, la robotique, la photographie numérique et les systèmes de sécurité. Il repose sur des opérations intensives en calcul, comme le **filtrage**, la **transformation**, et l'**analyse des images**, qui peuvent rapidement devenir coûteuses en temps d'exécution, surtout pour des images de haute résolution.

Le **multi-threading** offre une solution efficace en permettant de **paralléliser les calculs**, exploitant ainsi pleinement les capacités des **processeurs multicœurs modernes**. Par exemple, diviser une tâche en plusieurs threads permet de **traiter différentes parties** de l'image **simultanément**, réduisant ainsi le temps global d'exécution.

L'objectif de ce projet est d'**étudier l'impact** du **multithreading** sur les performances des algorithmes de traitement d'images, d'implémenter et d'optimiser des filtres et transformations de base, et de comparer les résultats avec des versions **séquentielles** en termes de **rapidité** et de **qualité**.

II. CONCEPTS ET DOCUMENTATION

a. Concepts de base du traitement d'images

Le traitement d'images repose sur des techniques visant à améliorer ou à analyser des images numériques. Parmi les opérations les plus courantes :

- **Filtrage** : Utilisé pour réduire le bruit ou adoucir les variations brusques dans une image. Exemples :
 - *Flou gaussien* : Moyennage des pixels pour lisser l'image.
 - *Filtre médian* : Remplace chaque pixel par la médiane des pixels voisins pour réduire le bruit impulsif.
- **Détection de contours** : Extraction des bordures dans une image.
 - *Détection de Canny* : Identifie les contours à partir des gradients d'intensité.
 - *Filtre Sobel* : Calcule les dérivées pour détecter les changements d'intensité.
- **Transformations** : Modifications géométriques ou fréquentielles.
 - *Transformation de Fourier* : Analyse les composantes fréquentielles d'une image pour le filtrage fréquentiel.
 - *Redimensionnement et rotation* : Changements de taille et d'orientation.

b. Concepts du multi-threading

Le multi-threading consiste à exécuter plusieurs tâches simultanément en partageant les ressources d'un même processus. En C++, la gestion des threads repose sur les éléments suivants :

- **std::thread** : Permet de créer et gérer des threads.
- **std::mutex** : Garantit un accès exclusif à une ressource partagée. Exemple : éviter que deux threads modifient la même variable simultanément.
- **std::lock_guard** : Simplifie la gestion des mutex en verrouillant et en déverrouillant automatiquement.
- **Variables de condition** : Synchronisent les threads en leur permettant de s'attendre mutuellement à l'aide de conditions spécifiques.

Les défis liés au multi-threading incluent :

- **Synchronisation** : Coordination entre threads pour éviter les conflits ou les erreurs.
- **Granularité des tâches** : Diviser une tâche en segments suffisamment petits pour équilibrer la charge entre les threads sans surcoût excessif.

c. Bibliothèques utilisées

OpenCV : Une bibliothèque open-source largement utilisée pour le traitement d'images. Elle offre des fonctionnalités pour la lecture, l'écriture, et la manipulation d'images, ainsi que des outils avancés comme la détection de contours et les transformations fréquentielles.

Standard Library C++ : Fournit des outils natifs pour le multi-threading (std::thread, std::mutex, etc.) et pour la gestion de la mémoire, des données, et des exceptions.

III. DÉVELOPPEMENT ET IMPLÉMENTATION

a. Implémentation séquentielle

Ce projet étant petit et utilisant uniquement **OpenCV**, il n'y a pas besoin d'une algorithmie complexe. Nous avons un dossier *src* et un autre *include*, et nous avons choisi cette arborescence:

```
#CPMULTI/

├── CMakeLists.txt    # Project builder config
├── README.md
├── include/          # Header files (.h)
│   ├── ImageLoader.h
│   ├── Filters.h
│   ├── ImageSplitter.h
│   ├── PerformanceAnalyzer.h
│   └── ThreadPool.h
├── src/              # Source files (.cpp)
│   ├── main.cpp
│   ├── ImageLoader.cpp
│   ├── Filters.cpp
│   ├── ImageSplitter.cpp
│   ├── PerformanceAnalyzer.cpp
│   └── ThreadPool.cpp
├── data/             # Images
│   ├── input.jpg
│   └── output.jpg
└── docs/             # Documentation
    └── rapport.pdf
```

Les **classes** des différentes opérations d'OpenCV contiennent uniquement des fonctions **statiques** afin de pouvoir les appeler lorsque cela est nécessaire. Pour la classe de l'analyseur, nous l'instancierons dans `main` et la passerons en argument tout au long des fonctions.

Nous utilisons une **`unordered_map`** dans la classe **`PerformanceAnalyzer`** pour stocker les temps enregistrés, optimisant ainsi les performances grâce à des accès **rapides** en **$O(1)$** .

Par la suite, nous avons ajouté une classe **`Menus`** afin de nous permettre de choisir entre chaque **filtre** ou **transformation**.

d. Optimisation avec le multi-threading

L'**objectif** de cette implémentation est **d'accélérer** le traitement d'une image en appliquant un filtre en **parallèle** sur **quatre** sous-parties de l'image après l'avoir **divisée**.

L'utilisation du multi-threading permet **d'exploiter** les cœurs multiples du processeur, **réduisant** ainsi le **temps** total de **calcul**.

Tout d'abord, nous découpons l'image en **quatre** parties, puis nous instancions une liste de threads, où chaque **thread** s'occupera d'appliquer le **filtre** sur sa région de l'image. Ensuite, nous attendons que tous les threads terminent avant **d'assembler** l'image finale.

Dans notre projet, nous aurions pu créer une pool de threads à l'exécution du programme afin d'éviter d'instancier et de détruire les threads à chaque fois.

IV. ANALYSE DES PERFORMANCES

a. Chargement de différentes images

Nous allons tout d'abord mesurer le temps que notre programme prend à charger une image légère et une image lourde.



La **première** photo pèse **462 Ko** (soit 473 831 octets, environ 0,45 Mo), tandis que la deuxième pèse **11,2 Mo** (soit 11 786 595 octets). La différence entre les deux est de 11 312 764 octets (environ **10,79 Mo**), ce qui représente une réduction d'environ **96 %** par rapport à la taille de la deuxième photo.

Pour la première photo le temps de chargement est de **12.3267 ms**.

Pour la deuxième photo le temps de chargement est de **801.518 ms**.

(**Avertissement** : la première photo est sur format **png** et la deuxième est au format **jpg**. Il pourrait avoir un **décalage** suivant comment **openCV** se comporte avec les différents formats mais **nous allons le négliger**)

La différence de temps est de **789,1913 ms** soit une différence de **98,46 %**.

Dans notre cas, nous pouvons dire que le **temps augmente** à peu près de façon **proportionnelle** en fonction de la **taille de l'image**.

(**Avertissement** : les mesures **varient** pour la même image sur **différents lancements** du script. Par manque de temps nous n'avons pas fait plusieurs mesures et ni pris la moyenne de celles-ci. Pour être plus précis c'est conseillé de réaliser ces mesures, dans notre cas nous avons considéré la première mesure.)

b. Performances version séquentielle

Voici les valeurs enregistrées pour la version séquentielle du programme:

	Denoising	Invert Color	Median	Gaussian	Sobel	Canny	Rotate	FT	
Photo 1	14335.6	0.3391	68.1661	9.5659	14.2752	15.6126	14.955	24.4004	ms

Nous pouvons observer que le filtre médian est celui qui prend le plus de temps à traiter, ensuite Canny. Le plus rapide est l'inversion des couleurs.

Ici un classement en temps nécessaire décroissant :

- 1 - Denoising
- 2 - Médian
- 3 - FT
- 4 - Canny
- 5 - Rotate
- 6 - Sobel
- 7 - Gaussian
- 8 - Invert Color

c. Performances version multi-thread

Voici les valeurs enregistrées pour la version multithread du programme:

- TEMPS GLOBAL :

	Denoising	Invert Color	Median	Gaussian	Sobel	Canny	Rotate	FT	
Photo 1	7870.48	1.3387	19.1685	10.1752	5.175	15.4288	8.498	6.956	ms

- TEMPS PAR THREAD POUR LA PHOTO 1 (plus petite) :

	Denoising	Invert Color	Median	Gaussian	Sobel	Canny	Rotate	FFT	
Thread 1	7787.6	0.4074	16.6115	9.0113	4.1166	14.0921	7.6128	5.7796	ms
Thread 2	7835.95	0.4038	16.9702	9.1453	3.768	14.4456	7.3974	5.7782	ms
Thread 3	7869.22	0.396	16.8607	9.1641	4.1872	13.1174	7.3209	5.8438	ms
Thread 4	7799.5	0.3883	17.8675	9.2002	3.8411	14.2477	7.4816	5.8385	ms
Image Split	0.0316	0.0338	0.0302	0.0363	0.0314	0.0463	0.0341	0.0515	ms

Ici un classement en temps nécessaire décroissant :

- 1 - Denoising
- 2 - Médian
- 3 - Canny
- 4 - Gaussian
- 5 - Rotate
- 6 - FT
- 7 - Sobel
- 8 - Invert Color

V. RÉSULTATS ET DISCUSSION

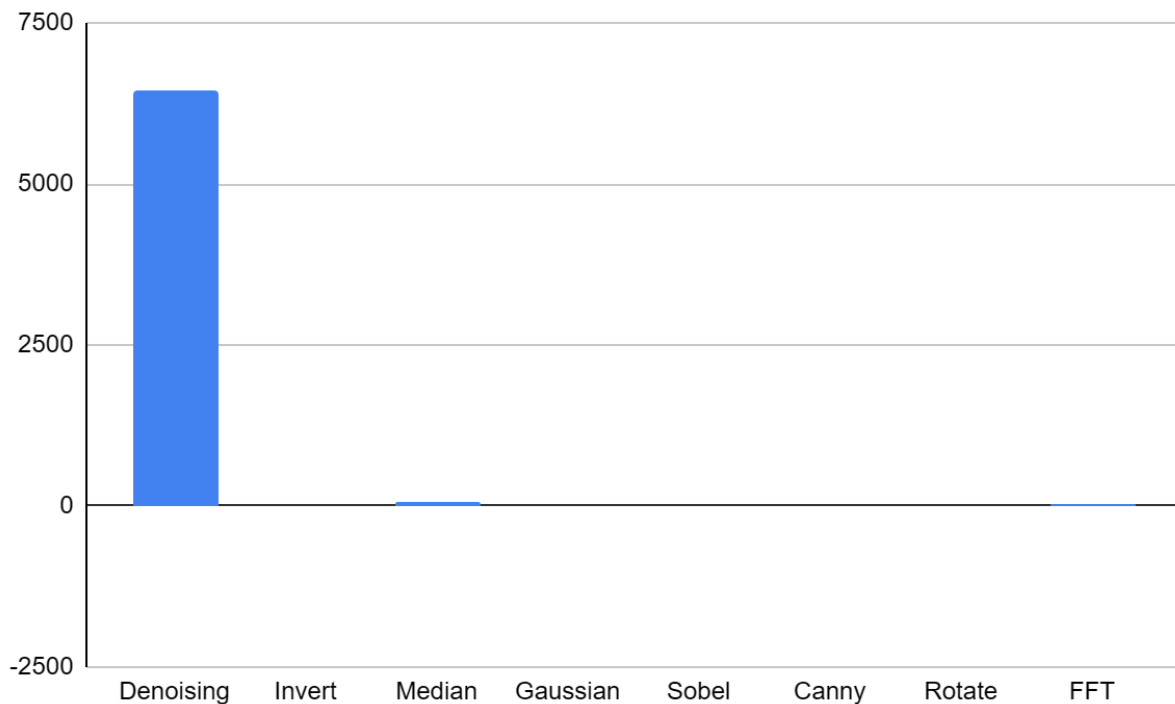
a. Résumé des observations - comparaison

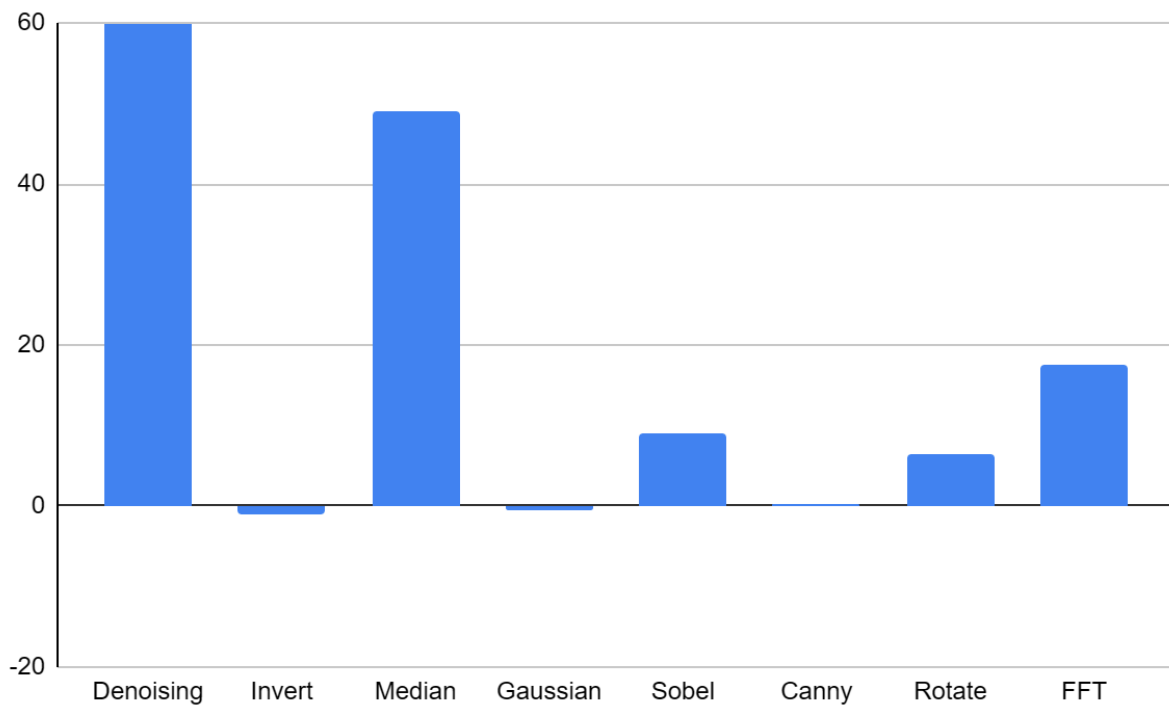
	Denoising	Invert Color	Median	Gaussian	Sobel	Canny	Rotate	FFT	
Séquentiel	14335.6	0.3391	68.1661	9.5659	14.2752	15.6126	14.955	24.4004	ms
Thread	7870.48	1.3387	19.1685	10.1752	5.175	15.4288	8.498	6.956	ms
Différence	6465,12	-0,9996	48,9976	-0,6093	9,1002	0,1838	6,457	17,4444	ms

Nous avons **gagné** énormément de **temps** sur le **débruitage** et le filtre **médian**.

En revanche, pour le filtre de **Canny**, la différence de performance n'a pas été significative.

Concernant l'inversion des **couleurs** et le filtre **gaussien**, l'utilisation des threads s'est avérée **défavorable**. Toutefois, notons que la perte de temps est inférieure à **1 ms**.





Sur le premier graphique, nous constatons clairement que le **débruitage** (denoising) est beaucoup plus **rapide** grâce au **multi-threading**.

Si nous réduisons l'échelle en ignorant le débruitage, nous observons que la **perte** de temps sur certains filtres est **minime** par rapport au **gain global** obtenu.

Le temps global gagné sur l'ensemble des filtres et transformations est de 6545,69 ms, soit **6,54 secondes**.